

1. 二进制/十六进制/十进制换算

- 分组法：二进制每 4 位一组 → 十六进制；例： $0101\ 1100_2 = 0x5C$
- 加权法：二进制 → 十进制：按位相加 $\sum bit_i \cdot 2^i$ ；例： $10111011_2 = 187$

2. 位运算 vs 逻辑运算 (C)

- 位运算 (逐位)：& | ^ ~ << >>
- 逻辑运算 (真值)：! && ||
 - !x：零→1，非零→0；结果类型是 int (只会是 0 或 1)。
 - !!x：把任意整数规范化为布尔值 0/1。

3. 有符号/无符号混合比较 (常规算术转换)

- 参与运算的有无符号整数同时存在 → 有符号数先转为无符号数再比较/运算。
- 典型坑： $-10 > 10u$ 为真，因为 $(unsigned)-10 = 2^w - 10$ (很大)。

4. 补码、符号扩展与字节序

- short → int：符号扩展 (复制最高位符号位)。
例：short -12 = 0xFFFF4 → int 0xFFFFFFFF4
- 小端序 (x86)：低有效字节放在低地址。
因此读取“首字节 (最低地址字节)”时，看数的最低 8 位。

5. IEEE-754 单精度 (float)

- 字段：[1-bit 符号 s][8-bit 阶码 exp][23-bit 尾数 frac]
 - 偏置 (Bias) = 127
 - 规格化： $1 \leq M < 2$ ， $M = 1 + frac/2^{23}$ ， $E = exp - Bias$
- 特殊值：
 - $+\infty$ ：s=0, exp=0xFF, frac=0 → 0x7F800000
 - $-\infty$ ：s=1, exp=0xFF, frac=0
 - NaN：exp=0xFF, frac≠0
- 十六进制 → 数值流程：
 1. 拆字段：s | exp | frac
 2. 若 $0 < exp < 255$ ：规格化： $M = 1 + frac/2^{23}$ ， $E = exp - 127$
 3. 拼： $v = (-1)^s \cdot M \cdot 2^E$
例：0xA1100000 → s=1, exp=0x42=66 → E=-61, frac=0x100000 → $M = 1 + 1/8 = 1.125$

6. “就近取偶”舍入 (Round to nearest, ties to even)

- 只保留 k 位 (如小数点后 3 位):
 - 设被截断段前三位为 **G** (Guard)、**R** (Round)、**S** (Sticky=其余位 OR)。
 - 规则:
 - $G=0$ → 直接截断 (向下)。
 - $G=1$ 且 $(R|S)=1$ → 进位。
 - 平局: $G=1, R=0, S=0$ → 看保留部分 LSB: 偶则不进位, 奇则进位。
- 判断是否能取别的值: 先算出候选相邻值 (间隔 $ULP=2^{\{-k\}}$), 比较距离; 与原数最近且“偶”的那个胜出。

7. 浮点运算的精确性与结合律

- double 可精确表示所有 $|x| < 2^{53}$ 的整数与其相加/减的结果, 只要中间不超出精确整数范围。
- 加法在此范围内可视为结合 (例如把 int 转 double 后相加, 值域不大时)。
- 乘法通常不结合: 中间结果易超 2^{53} , 产生舍入, 故 $(dx*dy)*dz$ 未必等于 $dx*(dy*dz)$ 。
- NaN 规则: NaN 不等于任何值 (包括自身); 例如 $0.0/0.0$ 。

8. x86-64 寄存器名映射

- 64 位: `rax rbx rcx rdx rsi rdi rsp rbp ...`
- 32 位低位: `eax ebx ecx edx ...`
- 16/8 位子寄存器: `ax/al/ah, bx/bl/bh, cx/cl/ch, dx/dl/dh ...`
- 题目考点: `rbx` 低 32 位 → `ebx`; `rcx` 最低字节 → `cl`

9. 有效地址 (AT&T 语法)

- 形式: `disp(base, index, scale)`, 含义:
 $EA = disp + base + index \times scale$ ($scale \in \{1, 2, 4, 8\}$; base 或 index 可省略为 0)。
- 不涉及大小端 (只是地址算术)。
- 例题 (`rdx=0xC000, rcx=0x0200`)
 - `0x10(%rdx,%rcx,4)` → $0x10 + 0xC000 + 0x0200*4 = 0xC810$
 - `0x30(,%rdx,8)` → $0x30 + 0xC000*8 = 0x60030$

10. 指令访问计数示例 (movq 交换两块内存)

```
movq (%rdi), %rax    ; 1 次内存读
movq (%rsi), %rdx    ; 1 次内存读
movq %rdx, (%rdi)    ; 1 次内存写
movq %rax, (%rsi)    ; 1 次内存写
```

- 内存访问: 4 次
- 通用寄存器读/写: 每条大约 2 次 (读+写), 合计 8 次 (不计取指与标志位)。

易错清单

- `!!x` 的结果是 **int** 类型的 0/1。
- 混合比较：先把**有符号数转无符号** → 常见坑。
- `short→int`：**符号扩展**；看“首字节”要考虑**小端**。
- Float：记住 **Bias=127**，`+∞=0x7F800000`。
- 舍入：G-R-S + **就近取偶**；平局看 LSB 的“偶/奇”。
- Double 精度阈： **2^{53}** ；加法在此范围可“近似结合”，乘法不可。
- x86 有效地址： **$EA = disp + base + index \times scale$** ；`(%base,%index,scale)` 切勿把 base/index 反了。